

Practica 9 - Excepciones

1. Una excepción es una situación anómala que se da en la ejecución de un programa y que se supone que ocurre con poca frecuencia.
2. Para que un lenguaje trate excepciones debe proveer:

- Un modo de definir las
- Una forma de alcanzarlas, invocarlas
- Una forma de manejarlas
- Un criterio de continuación

No todos los lenguajes de programación proveen un sistema de excepciones, como Go o Rust que prefieren tratar a los errores como valores comunes y corrientes para hacer el código más predecible y seguro

3. Cuando un lenguaje no provee un mecanismo de excepciones como C, los errores no se pueden lanzar o atrapar automáticamente. Si ocurre un fallo grave, como acceder a memoria inválida o dividir por cero, el SO interviene y mata al proceso (Crash o Segmentation Fault). Para errores menos graves, como un archivo no encontrado, si el programador no escribe el código explícito para verificar cada paso, el programa continuará ejecutándose con datos inválidos. Si se puede simular, requiere resolver dos problemas:
 - Pasar la info del error hacia arriba en el programa
 - Alterar el flujo normal para saltar directamente al código que sabe cómo resolver el error

Por ejemplo, en vez de que una función devuelva el resultado, devuelve una estructura o tupla que contiene el resultado y un código de estado:

```
// Definimos una estructura para simular la "Excepción"
```

```
typedef struct {  
    int código;  
    char* mensaje;  
} Excepción;
```

```
// Estructura que retorna la función
```

```
typedef struct {  
    int resultado;  
    Excepción error;  
} ResultadoDivision;
```

```
ResultadoDivision dividir(int a, int b) {
```

```
    ResultadoDivision r;  
    if (b == 0) {  
        r.error.código = 404;
```

```

    r.error.mensaje = "División por cero detectada.";
    return r; // "Lanzamos" el error devolviéndolo
}
r.resultado = a / b;
r.error.codigo = 0; // 0 significa sin error (OK)
return r;
}

```

El programador esta obligado a evaluar el codigo de error después de llamar a la función mediante un if

4.

01. Modelos después de manejar una excepción:

1. Modelo de terminación: cuando se lanza una excepción, el codigo donde ocurrió el error se da por terminado. La ejecucion continua en la linea posterior al bloque de captura
2. Modelo de reanudación: el manejador de la excepción puede corregir el problema en plena ejecución y reanudar la ejecución en la misma instrucción que causo el error. La funcion se congelo mientras se resuelve la anomalia

02. Modelo de Terminación: Python

- a. Las excepciones son class que heredan de la clase BaseException o Exception
- b. Se lanza con la palabra "raise" seguida de un instancia de la excepción
- c. Se maneja mediante la estructura try..except. Se pueden tener muchos except
- d. Al terminar el except, el control sigue con la primera linea de codigo que este afuera y después del try-except. El bloque de codigo que fallo en try queda cancelado y no se reintenta nuevamente.

Modelo de reanudacion: PL/I o Smalltalk

- a. Se definen condiciones de error (ej. UNDERFLOW o personalizadas) asociadas a un manejador (ON condition)
- b. El SO lo lanza automáticamente ante un fallo o el programador mediante SIGNAL condition
- c. Se establece un bloque ON CONDITION previamente en el código. Cuando la condición se dispara, el flujo salta temporalmente a ese bloque
- d. El manejador puede alterar las variables que causaron el fallo, y regresar a la instrucción que fallo paa reintentarla

03. El modelo de reanudación es mas inseguro, complejo y propenso a errores:

- Si el manejador de la excepción no corrige el estado de la memoria a la perfección y el programa regresa a la instrucción, esta volverá a fallar

lanzando la excepción de nuevo y atrapando al sistema en un ciclo infinito de fallos

- Meter el flujo de ejecución en medio de una función que ya ha fallado asume que el contexto de las variables sigue siendo seguro. Un error intermedio suele dejar datos corruptos o a medio procesar en memoria. Causa comportamientos impredecibles

5. Cuando una excepción se lanza y la función actual no tiene un bloque para capturarla, la excepción comienza a propagarse hacia arriba en la pila de llamadas. Si llega al nivel mas alto y sigue sin encontrarse manejador, el lenguaje debe tomar una decisión por defecto.

- Python: cuando una excepcion no se captura, Python propaga el error dinamicamente hacia la funcion que la invoco. Si llega a la funcion principal sin ser atrapada, el interprete aborta la ejecucion del programa. Imprime el Traceback mostrando la secuencia de archivos y lineas de codigo por las que viajo la excepcion
- Java: divide las excepciones en dos grupos. Las Inchecked Exepctions, errores en tiempo de ejecucion como NullPointerException, se propaga dinamicamente como python. Las Checked Exceptions, como IOException, deben ser declaradas explicitamente en la firma del metodo usando "throws". Si una excepcion no capturada llega al metodo main, el thread (hilo) muere. Si ocurre en el hilo principal, el programa finaliza y la JVM imprime el Stack Trace
- PL/I: no utiliza una pila de llamadas moderna como los lenguajes OO. Utiliza condiciones. Cuando una condicion se dispara y no hay bloque ON, el entorno de ejecucion busca hacia atras en lobbloques de procedimientos activos. Si no se encuentra un manejador, ejecuta una accion por defecto del sistema llamada ERROR condition. El comportamiento estnadar de esta condicion es escribir un mensaje de diagnostico en el log y llamar a la instruccion FINISH lo que provoca la terminacion anormal del programa delegando el cierre al SO de la mainframe

6.